

Apache Spark Basics

Distributed Data Processing at Any Scale

■ APACHE SPARK

■ LEARNING OUTCOME

By the end of this module you will understand Spark's execution model, process DataFrames with PySpark API, write Spark SQL, optimise Spark jobs, and run Spark on Databricks or AWS EMR.

■ Skills You Will Gain

- Explain Spark architecture: driver, cluster manager, executors, partitions
- Use PySpark DataFrame API for transformations and aggregations
- Write Spark SQL on registered temporary views
- Understand lazy evaluation and the DAG execution model
- Optimise jobs: partitioning, caching, broadcast joins, Parquet
- Run Spark on Databricks, AWS EMR, or locally

■ Apache Spark is the industry standard for big data processing. It's on 80% of DE job descriptions and can process petabytes of data in minutes.

SECTION 1

Spark Architecture

Component	Role
Driver Program	The main process — creates SparkContext, builds the execution plan (DAG), schedules tasks
Cluster Manager	Allocates resources across the cluster — YARN (Hadoop), Kubernetes, Mesos, or Spark Standalone
Executor	Worker JVM processes on cluster nodes — execute tasks, store data in memory or on disk, communicate with the driver
Partition	A chunk of data assigned to one executor — the unit of parallelism. More partitions = more parallelism
Task	The smallest unit of work — one task processes one partition
Stage	A group of tasks that can run in parallel without a shuffle between them
DAG Scheduler	Converts your transformation code into a directed acyclic graph of stages — optimises execution

Lazy Evaluation — Spark's Key Design Principle

Transformations (filter, select, groupBy, join) are LAZY — they don't execute immediately. They build a logical plan (DAG) describing what to do. Actions (show, count, write, collect) are EAGER — they TRIGGER actual execution. Spark optimises the entire DAG before running — combining steps, pushing filters down, choosing join strategies. This enables powerful optimisations impossible in pandas or SQL.

SECTION 2

PySpark DataFrame API

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType

# Create or get existing SparkSession
spark = SparkSession.builder \
    .appName('SalesAnalysis') \
    .config('spark.sql.adaptive.enabled', 'true') \
    .config('spark.sql.shuffle.partitions', '200') \
    .getOrCreate()

# Read data
df = spark.read.parquet('s3://my-lake/sales/')

# Transformations (LAZY - nothing executes yet)
result = (df
    .filter(F.col('amount') > 0) # filter bad rows
    .withColumn('revenue', F.col('qty') * F.col('price')) # add column
    .withColumn('month', F.date_format('order_date', 'yyyy-MM'))
    .withColumn('year', F.year('order_date'))
    .groupBy('region', 'month') # aggregate
    .agg(
        F.sum('revenue').alias('total_revenue'),
        F.count('*').alias('order_count'),
        F.avg('revenue').alias('avg_order'),
        F.countDistinct('customer_id').alias('unique_customers')
    )
    .orderBy('month', F.col('total_revenue').desc()) # ACTION - triggers execution
)

result.show(20, truncate=False)
result.write.mode('overwrite').parquet('s3://my-lake/output/monthly_sales/')
```

SECTION 3

Spark SQL

```
# Register DataFrame as a temporary view for SQL queries
df.createOrReplaceTempView('orders')
customers_df.createOrReplaceTempView('customers')

# Run Spark SQL - full SQL support including window functions
monthly = spark.sql("""
    SELECT region, DATE_TRUNC('month', order_date) AS month,
    SUM(amount) AS revenue, COUNT(DISTINCT customer_id) AS unique_customers,
    SUM(amount) / COUNT(*) AS avg_order_value,
    -- Window function: running total
    SUM(SUM(amount)) OVER (PARTITION BY region ORDER BY DATE_TRUNC('month', order_date)) AS running_revenue
    FROM orders WHERE status = 'delivered' AND order_date >= '2024-01-01'
    GROUP BY 1, 2
    ORDER BY region, month
""")
monthly.show()
```

Performance Optimisation

Technique	How to Apply	Why It Helps
Use Parquet	Write with <code>.write.parquet()</code> , read <code>.read.parquet()</code>	Columnar, compressed — 10x faster than CSV
Partition data	<code>df.repartition(200)</code> or write with <code>.partitionBy('date')</code>	Parallelism + partition pruning on reads
Coalesce small	<code>df.coalesce(10)</code> before writing	Avoids many small files — kills performance
Cache hot DataFrames	<code>df.cache()</code> then <code>df.count()</code> to materialise	Avoid recomputing same data in multiple actions
Broadcast joins	<code>F.broadcast(small_df)</code> in join expression	Avoids expensive shuffle of large table
Avoid Python UDFs	Use built-in F.functions instead of Python lambda	Built-ins run in JVM — 10-100x faster than Python UDFs
Adaptive Query Exec	<code>spark.sql.adaptive.enabled = true</code> (default in Spark 3.0+)	Auto-optimises shuffle partitions and join strategies

```
# Broadcast join – ship small table to every executor
result = large_df.join(
    F.broadcast(small_lookup_df), # broadcast tables < 10MB on='product_id', how='left' ) # Cache
DataFrame used multiple times
df_filtered = df.filter(F.col('status') == 'delivered').cache()
df_filtered.count() # materialise the cache
monthly_agg = df_filtered.groupBy('month').agg(F.sum('revenue'))
region_agg = df_filtered.groupBy('region').agg(F.count('*')) # Write optimised output
result.repartition(50) \
.write \ .mode('overwrite') \ .partitionBy('year', 'month') \ .parquet('s3://bucket/output/')
```

■ PRACTICE Q & A

- Q1.** What is lazy evaluation in Spark and why is it important?

A: Transformations (filter, select, join) don't execute — they build a logical plan. Only actions (show, write, count) trigger execution. Allows Spark's Catalyst optimizer to optimise the full plan before running.
- Q2.** What is the difference between `repartition()` and `coalesce()`?

A: `repartition(n)`: full shuffle, can increase or decrease partitions. `coalesce(n)`: no shuffle, only decreases by merging. Use `coalesce` before writing to avoid many small output files.
- Q3.** When should you use `df.cache()` and when should you avoid it?

A: Use when the same DataFrame is referenced in multiple actions (e.g. two different aggregations). Avoid if the DataFrame is used only once — caching wastes memory and adds overhead.
- Q4.** What is a broadcast join and when does Spark apply it automatically?

A: Sends the small DataFrame to every executor, avoiding a costly shuffle of the large DataFrame. Spark auto-broadcasts tables under `spark.sql.autoBroadcastJoinThreshold` (default 10MB). Explicitly use `F.broadcast()` to force it.
- Q5.** Why is Parquet preferred over CSV for big data processing?

A: Parquet is columnar — analytics queries only read needed columns, not all. It compresses 5-10x better than CSV. It stores schema. Spark can skip entire row groups with partition pruning. CSV is text — slow to parse.

■ PySpark Cheat Sheet	
<code>SparkSession.builder.getOrCreate()</code>	Start Spark session
<code>spark.read.parquet('path')</code>	Read Parquet files
<code>spark.read.csv('path', header=True)</code>	Read CSV files
<code>df.filter(F.col('c') > 0)</code>	Filter rows
<code>df.withColumn('new', expr)</code>	Add / modify column
<code>df.groupBy('col').agg(F.sum())</code>	Group and aggregate

<code>df.join(df2, 'key', 'left')</code>	DataFrame join
<code>F.broadcast(small_df)</code>	Broadcast small table
<code>df.cache()</code>	Cache in memory
<code>df.repartition(200)</code>	Redistribute partitions
<code>df.coalesce(10)</code>	Reduce partitions (no shuffle)
<code>df.write.partitionBy('date').parquet</code>	Write partitioned Parquet